A projective-line code disproves conjecture 00000007973

asfigh665

October 15, 2026

Abstract

The conjectured minimum-weight identity $d = \deg D - g$ fails for a standard evaluation algebraic-geometric code on $\mathbf{P}^1_{\mathbf{F}_3}$. For $D = P_0 + P_1 + P_2$ and $G = P_\infty$, the code is the $[3, 2, 2]_3$ Reed–Solomon code, whereas the proposed identity gives $d = 3$. A dependency-free Lean 4 project checks the finite field, the complete evaluation code, its minimum nonzero weight, and its weight enumerator.

1 The assertion being refuted

Conjecture 00000007973 concerns the weight enumerator of a code $C(X, D, G)$ from algebraic geometry. Both language versions explicitly assert that its minimum weight is $\deg D - g$, where g is the genus of X. As usual, minimum weight means the minimum over *nonzero* codewords. We disprove this clause; consequently the conjunction of assertions in the conjecture is false, independently of its proposed modular-form description.

We use the standard evaluation AG code

$$C_L(D, G) = \{(f(P_0), f(P_1), f(P_2)) : f \in L(G)\}, \qquad L(G) = \{0\} \cup \{f \in \mathbf{F}_3(X)^\times : (f) + G \geq 0\}.$$

Here $X = \mathbf{P}^1_{\mathbf{F}_3}$, t is its affine coordinate, P_j is the rational point $t = j$, $D = P_0 + P_1 + P_2$, and $G = P_\infty$. The curve is smooth, projective, and geometrically integral, with genus $g = 0$. The evaluation points are distinct, and the supports of D and G are disjoint. Also $0 < \deg G = 1 < \deg D = 3$ and $2g - 2 < \deg G$, so the example satisfies the usual nondegenerate evaluation-code hypotheses.

Lemma 1. $L(P_\infty) = \{at + b : a, b \in \mathbf{F}_3\}$.

Proof. A rational function on the projective line with no finite poles is a polynomial in t. Indeed, write it as $p(t)/q(t)$ in lowest terms; any nonconstant irreducible factor of q supplies a finite closed point where the function has a pole. Thus q must be constant. In the local parameter $u = 1/t$ at infinity, a nonzero polynomial of degree m has pole order m. The bound of one on that pole order is therefore exactly $\deg f \leq 1$. The zero function is included separately. □

In particular the full Weierstrass semigroup at P_∞ is $\mathbb{N}_0$: the function t^m has its unique pole there, of order m. Its gap set is empty. Thus the semigroup data mentioned in the conjecture are completely specified in this example.

2 Exact minimum weight

The lemma gives the whole code explicitly:

$$C_L(D, G) = \{(b, a + b, 2a + b) : a, b \in \mathbf{F}_3\}.$$

The parameterization is injective: the value at 0 recovers b, and the value at 1 then recovers a. Hence this is a length-three, dimension-two linear code over $\mathbf{F}_3$.

Proposition 1. *The minimum nonzero Hamming weight of $C_L(D, G)$ is 2, and its weight enumerator is $1 + 6z^2 + 2z^3$.*

Proof. When $a = b = 0$, the word has weight zero. When $a = 0$ and $b \neq 0$, the word is a nonzero constant and has weight three; there are two such words. When $a \neq 0$, the polynomial $at + b$ has exactly one root in $\mathbf{F}_3$. All three field elements are evaluated, so its word has exactly two nonzero entries; there are $2 \cdot 3 = 6$ such words. In particular, $f = t$ gives the word $(0, 1, 2)$ of weight two. These cases exhaust all nine codewords, proving both claims. □

Thus

$$\boxed{d(C_L(D, G)) = 2 \neq 3 = \deg D - g.}$$

This discrepancy is not the zero codeword or an exceptional divisor issue. The familiar designed-distance lower bound is $d \geq \deg D - \deg G$; it is an equality in this example. The conjecture instead subtracts the genus, omitting the divisor G on which the evaluation space depends.

3 Lean verification and its scope

The accompanying project uses Lean 4.24.0 and its standard library only. `F3` is `Fin 3`, with addition and multiplication modulo three; `field_laws` checks the field identities and multiplicative inverses. `evaluate` constructs $t \mapsto at + b$ using those operations. `InCode` quantifies over all a, b, and `MinimumNonzeroWeight` requires both a lower bound for every nonzero codeword and an attaining codeword. The principal results are:

- `evaluation_linear` and `evaluate_injective` verify the algebraic parameterization;
- `minimum_weight_two` and `not_minimum_weight_three` establish the minimum and refute the claimed value;
- `complete_weight_enumerator` checks all nine parameter pairs;
- `counterexample` refutes the required genus-zero, degree-one instance; the final implication theorem explicitly transfers this contradiction to any assertion implying that instance.

All finite decisions are kernel checked. The project does not formalize schemes or general Riemann–Roch theory; the identification of this concrete evaluation space with $L(P_\infty)$, and the standard genus of $\mathbf{P}^1$, are the mathematical bridge supplied above. No general algebraic-geometry theorem is asserted as an added axiom in Lean. The checks therefore cover the full finite code and the contradiction, rather than a hard-coded comparison of two unexplained integers.

Reproduction. From `lean4/`, run `lake build`, `lake env lean Conjecture7973.lean`, and `lake exe verify`. The displayed axiom dependencies of the main results are only `propext` and `Quot.sound`. The project has no external packages, proof holes, native decision shortcuts, or custom axioms.